

Miscellaneous minor fixes for chrono

Contents

- [Introduction](#)
- [Wording](#)

Introduction

This is a collection of minor fixes and upgrades to the <chrono> library that have come to my attention since the acceptance of [P0355r7](#).

Wording

1. Feedback from the field asks for a way to identify if a `utc_time` represents a leap second. This proposed API is the only API that has field experience in providing that information in a way that is efficient, and provides all information the client desires when making a query in this area. It is used in the implementation of the conversion between `utc_time` and `sys_time`, and in the parsing and formatting of `utc_time`, so is likely to be present in any event. If we choose to specify it, it can be spelled without underscores, and be available to clients of <chrono>.

Insert new paragraphs in 26.7.2.3 Non-member functions [`time.clock.utc.nonmembers`]:

```
template <class Duration>
pair<bool, seconds>
is_leap_second(utc_time<Duration> const& ut)
```

Returns: A `pair<bool, seconds>` where `first` is true if `ut` is during a leap second insertion, and otherwise false. `second` is the number of leap seconds between 1970-01-01 and `ut`. If `first` is true, the leap second referred to by `ut` is included in the count.

2. Feedback from the field asks for a way to customize the spacing between a duration's value and its unit (e.g. supply a space, or a Unicode custom space between the value and the unit). This item proposes `%Q` to represent the durations's value and `%q` to represent the duration's unit, for formatting only. Example: `format("%Q %q", 45ms) == "45 ms"`.

Add two new rows to Table 87 — Meaning of format conversion specifiers:

%Q	The duration's numeric value (as if extracted via <code>.count()</code>).
%q	The duration's unit suffix as specified in <code>[time.duration.io]</code> .

3. About half of the clients are upset that the currently specified encoding for weekday implies that Sunday is the first day of the week (consistent with the current C and C++ specifications for `tm.tm_wday`), and the other half of the clients will be upset if the weekday encoding follows the ISO specification of [1, 7] maps to [Monday, Sunday].

This change strikes a compromise in an attempt to please everyone (a nearly impossible task).

- The `weekday{unsigned}` constructor accepts both mappings, which means that [0, 6] maps to [Monday, Saturday] **and** [1, 7] maps to [Monday, Sunday]. This is possible by simply accepting [0, 7] where [1, 6] maps to [Monday, Saturday] and both 0 and 7 map to Sunday.
- The explicit conversion to unsigned is removed from weekday and named conversions are inserted in its place: `c_encoding()` and `iso_encoding()`. The client can choose which mapping from weekday to unsigned he desires by choosing one of these member functions.

Modify 26.8.6.2 `[time.cal.wd.members]` as indicated:

```
constexpr explicit weekday(unsigned wd) noexcept;
```

Effects: Constructs an object of type `weekday` by initializing `wd_` with `wd`, except if `wd == 7`, stores 0. The value held is unspecified if `wd` is not in the range [0, 255].

~~constexpr explicit operator unsigned() const noexcept;~~

~~*Returns:* `wd_`.~~

constexpr unsigned c_encoding() const noexcept;

Returns: `wd_`.

constexpr unsigned iso_encoding() const noexcept;

Returns: `unsigned{((wd_ == 0u) ? 7u : wd_)}`.

4. `link` is missing some comparison operator definitions. The declarations are correctly in the synopsis. The missing definitions should be the "usual" formula based on `==` and `<`.

Add paragraphs to 26.10.9.3 Non-member functions `[time.zone.link.nonmembers]`:

```
bool operator!=(const link& x, const link& y) noexcept;
```

Returns: `!(x == y)`.

```
bool operator> (const link& x, const link& y) noexcept;
```

Returns: $y < x$.

```
bool operator<=(const link& x, const link& y) noexcept;
```

Returns: $!(y < x)$.

```
bool operator>=(const link& x, const link& y) noexcept;
```

Returns: $!(x < y)$.